Extended Abstract Track

Weight-space gradient descent induces approximate activity descent, making learning rules testable

Editors: List of editors' names

Abstract

A central question in neuroscience is whether biological neural networks optimise a loss function by adjusting synaptic weights – and if so, by what rule. Candidate rules are difficult to test because synaptic weights cannot be measured at scale in behaving animals, whereas neural *activities* can be recorded across learning. This raises the question: if a network performs gradient descent on its weights, what should we see in its activities? We show that weight-space gradient descent induces kernel-filtered activity descent; when the kernel is diagonal-dominant, each neuron approximately follows its own loss gradient. This yields a prediction testable with existing experimental techniques.

Keywords: neural activity, gradient descent, learning rules, computational neuroscience

1. Weight-space descent and observable activity

Gradient descent on the weights of a neural network embodies a simple learning principle: *the more that strengthening a synapse would improve downstream performance, the more that synapse is strengthened* (and vice versa—harmful synapses are weakened). We consider what this implies for the *observable* neural activations of the network.

Setup and derivation. Consider a feedforward network with layers $\ell = 1, \dots, L$, weights $\mathbf{W}$, and neural activities $A_i^{(\ell)}(\mathbf{W})$, trained to minimise a loss $\mathcal{L}$. Let $\mathcal{N}_\ell := \{1, \dots, n_\ell\}$ index the neurons in layer ℓ, let $\mathcal{P}_r$ index the scalar weights in layer r, and define the upstream parameter-index set $\mathcal{P}_{\le \ell} := \bigcup_{r=1}^{\ell} \mathcal{P}_r$. Thus $i, j, k \in \mathcal{N}_\ell$ index neurons in a fixed layer, whereas $\alpha \in \mathcal{P}_{\le \ell}$ indexes a scalar upstream weight W_α. We write $\mathbf{W}_{\le \ell} := (W_\alpha)_{\alpha \in \mathcal{P}_{\le \ell}}$ for the corresponding block of weights.

In gradient descent, every weight is updated simultaneously:

$$\Delta W_\alpha = -\eta \, \frac{\partial \mathcal{L}}{\partial W_\alpha}, \qquad \forall\, \alpha \in \mathcal{P}_{\le L}. \tag{1}$$

What is the induced change in the activity of neuron i in layer ℓ? Because $A_i^{(\ell)}$ depends only on $\mathbf{W}_{\le \ell}$, if the activity map is twice differentiable with bounded second derivatives along the update segment, Taylor expansion around the pre-update weights $\mathbf{W}_t$ gives

$$\Delta A_i^{(\ell)} \;:=\; A_i^{(\ell)}(\mathbf{W}_t + \Delta \mathbf{W}) - A_i^{(\ell)}(\mathbf{W}_t) \;=\; \nabla_{\mathbf{W}_{\le \ell}} A_i^{(\ell)}(\mathbf{W}_t)^\top \, \Delta \mathbf{W}_{\le \ell} + O\big(\|\Delta \mathbf{W}_{\le \ell}\|^2\big)\,.$$

Since $\Delta \mathbf{W} = O(\eta)$, substituting $\Delta W_\alpha = -\eta \, \partial \mathcal{L} / \partial W_\alpha$ gives

$$\Delta A_i^{(\ell)} \;\approx\; -\eta \sum_{\alpha \in \mathcal{P}_{\le \ell}} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \, \frac{\partial \mathcal{L}}{\partial W_\alpha}\,, \tag{2}$$

where all derivatives are evaluated at $\mathbf{W}_t$ and the omitted remainder is $O(\eta^2)$. For piecewise-linear activations such as ReLU, this statement applies when the step is small enough that no unit changes activation status, so the network remains within a single smooth piecewise-polynomial region of weight space. In a sequential feedforward network (one without skip or residual connections), the effect of any weight indexed by $\mathcal{P}_{\leq \ell}$ on the loss passes entirely through the activities of layer ℓ. This allows us to use the chain rule to write the effect on the loss, $\partial \mathcal{L}/\partial W_\alpha$, in terms of the activities at layer ℓ:

$$\Delta A_i^{(\ell)} \;\approx\; -\eta \sum_{\alpha \in \mathcal{P}_{\leq \ell}} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \left[\sum_{k \in \mathcal{N}_\ell} \left(\partial \mathcal{L}/\partial A_k^{(\ell)}\right) \left(\partial A_k^{(\ell)}/\partial W_\alpha\right) \right], \tag{3}$$

Exchanging the sums yields

$$\boxed{\Delta A_i^{(\ell)} \;\approx\; -\eta \sum_{k \in \mathcal{N}_\ell} \Phi_{ik}^{(\ell)} \, \frac{\partial \mathcal{L}}{\partial A_k^{(\ell)}}}, \tag{4}$$

where

$$\Phi_{ik}^{(\ell)} \;:=\; \sum_{\alpha \in \mathcal{P}_{\leq \ell}} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial A_k^{(\ell)}}{\partial W_\alpha} \;=\; \nabla_{\mathbf{W}_{\leq \ell}} A_i^{(\ell)} \cdot \nabla_{\mathbf{W}_{\leq \ell}} A_k^{(\ell)} \tag{5}$$

is a Gram matrix defined on the activities within each layer, and $\partial \mathcal{L}/\partial A_k^{(\ell)}$ is the full backpropagated loss gradient – a sufficient statistic that compresses everything downstream layers contribute to the weight updates affecting layer ℓ. In vector form, $\Delta \mathbf{A}^{(\ell)} \approx -\eta \, \Phi^{(\ell)} \, \nabla_{\mathbf{A}^{(\ell)}} \mathcal{L}$: gradient descent in weight space induces *kernel descent* within each layer of activity space. It holds for any differentiable sequential feedforward network regardless of depth, width, or activation function. Architectures with skip connections require additional treatment, as weights can influence the loss through paths that bypass intermediate layers.

Relation to the neural tangent kernel. The first-order approximation we study should be familiar to those acquainted with the neural tangent kernel (NTK) (Jacot et al., 2018). The kernel $\Phi^{(\ell)}$ is a layer-wise analogue of the empirical neural tangent kernel (NTK) for finite-width networks (Lee et al., 2020).

Self-terms drive each neuron toward its loss gradient; cross-terms interfere. The diagonal element $\Phi_{ii}^{(\ell)} = \|\nabla_{\mathbf{W}_{\leq \ell}} A_i^{(\ell)}\|^2$ collects contributions from *all* weights that influence neuron i. Because $\Phi_{ii}^{(\ell)} \geq 0$, the self-term update is aligned with $-\partial \mathcal{L}/\partial A_i^{(\ell)}$: each such neuron's activity moves down its own loss gradient, with an effective learning rate $\eta \, \Phi_{ii}^{(\ell)}$ that grows with the depth and width of its input pathways. Dead ReLU units have $\Phi_{ii}^{(\ell)} = 0$.

The off-diagonal elements $\Phi_{ik}^{(\ell)} = \nabla_{\mathbf{W}_{\leq \ell}} A_i^{(\ell)} \cdot \nabla_{\mathbf{W}_{\leq \ell}} A_k^{(\ell)}$ for $k \neq i$ couple neurons *within the same layer*. Two neurons whose activities depend on overlapping upstream weights can have large $|\Phi_{ik}^{(\ell)}|$, whereas neurons supported by effectively orthogonal parameter directions have $\Phi_{ik}^{(\ell)} \approx 0$. These cross terms need not align with $-\partial \mathcal{L}/\partial A_i^{(\ell)}$. Whether the self-term or the cross-terms dominate determines whether activity changes look like activity-space gradient descent.

Approximate kernel diagonality. Cross terms are small when different neurons' activities depend on nearly orthogonal directions in upstream weight space. Width can increase each neuron's private incoming parameters, but shared upstream weights mean that activity gradients need not become independent. Depth can strengthen these shared correlations, while learning may decorrelate effective pathways. The experiments below quantify how width and depth affect the resulting approximation.

Activity descent as a proxy for weight descent. Combining the kernel identity (Equation (4)) with diagonal dominance yields a simple approximation:

$$\Delta A_i^{(\ell)} \;\approx\; -\eta\, \Phi_{ii}^{(\ell)}\, \frac{\partial \mathcal{L}}{\partial A_i^{(\ell)}}\,. \tag{6}$$

Just as weight descent says *strengthen each synapse in proportion to how much it helps*, activity descent says *increase each neuron's firing in proportion to how much that increase would help* (and decrease it when it hurts). Equation (6) therefore identifies a useful approximation regime: when $\Phi^{(\ell)}$ is sufficiently diagonal-dominant, each neuron's activity moves in the direction that locally reduces the loss, up to a neuron-specific scaling factor $\Phi_{ii}^{(\ell)}$.

A testable prediction for neuroscience. If a biological circuit performs a gradient-like synaptic update in a diagonal-dominant regime, Equation (6) predicts that each active neuron's change is sign-aligned with its negative loss gradient. This can be tested by recording population activity for matched task conditions across learning, estimating $\partial \mathcal{L}/\partial A_i^{(\ell)}$ with a task-fitted model (Richards and Kording, 2023), and measuring sign agreement or rank correlation with $\Delta A_i^{(\ell)}$. Matching conditions isolate learning from changes in inputs or behavioural state; the unknown $\Phi_{ii}^{(\ell)}$ absorbs per-neuron gains, so exact magnitudes are not predicted. Robust misalignment would challenge the fitted gradient, the gradient-like update, or the diagonal-dominance assumption.

Activity descent is necessary but not sufficient for weight descent. Observing that activities follow their loss gradient constrains the projection of $\Delta \mathbf{W}$ onto the row space of the activity Jacobian $\partial \mathbf{A}/\partial \mathbf{W}$, but says nothing about the (typically vast) null space of weight changes that leave activities unchanged on the current input. Many weight updates can induce the same observed $\Delta \mathbf{A}$.

Experiments. We test Equations (4) and (6) in a 3-hidden-layer ReLU MLP trained by online SGD ($\eta = 0.005$) on 40 binary 4×4 bar images (20 per class; input dimension 16; output dimension 2). Figure 1 compares predicted and measured single-step activity changes at widths 8 and 48, follows training at both widths, then sweeps width from 4 to 512 at fixed depth 3 and depth from 2 to 8 at fixed width 32. Each sweep uses three random seeds trained for 2000 SGD steps.

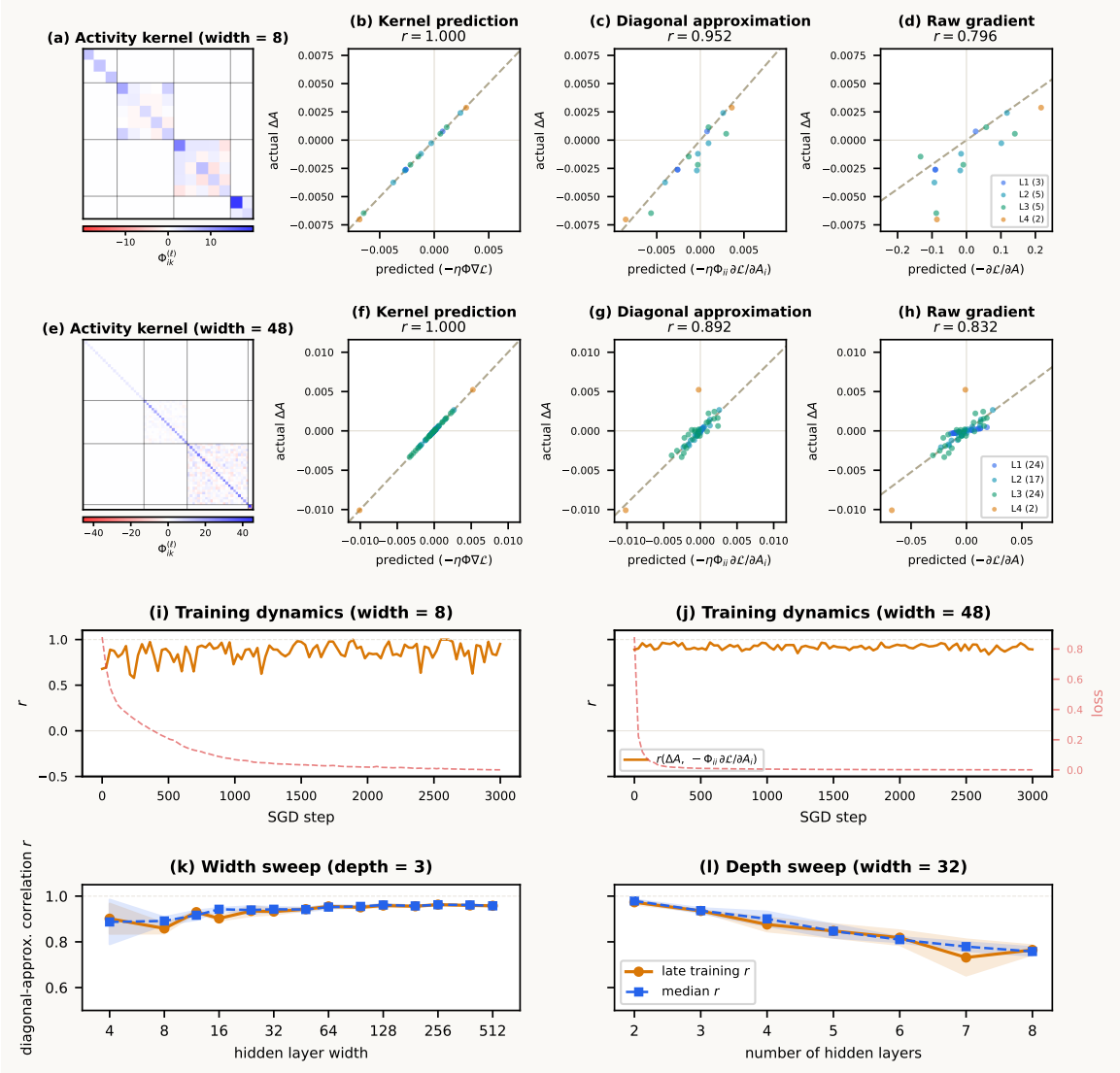

Figure 1: **Kernel descent and its diagonal approximation.** (a–d) Width 8 and (e–h) width 48: the full kernel predicts the activity update exactly, while diagonal scaling improves on the raw gradient. (i–j) Diagonal-approximation correlation and loss through training. (k–l) Accuracy across width and depth; orange shows the final-five mean, blue the training median, and bands ±1 s.d. across three seeds. Dead ReLU units are excluded.

References

Arthur Jacot, Franck Gabriel, and Clément Hongler. Neural Tangent Kernel: Convergence and Generalization in Neural Networks, June 2018.

Jaehoon Lee, Lechao Xiao, Samuel S. Schoenholz, Yasaman Bahri, Roman Novak, Jascha Sohl-Dickstein, and Jeffrey Pennington. Wide Neural Networks of Any Depth Evolve as Linear Models Under Gradient Descent. *Journal of Statistical Mechanics: Theory and Experiment*, 2020(12):124002, December 2020. ISSN 1742-5468. doi: 10.1088/1742-5468/abc62b.

Blake Aaron Richards and Konrad Paul Kording. The study of plasticity has always been about gradients. *The Journal of Physiology*, 601(15):3141–3149, 2023. ISSN 1469-7793. doi: 10.1113/JP282747.

Appendix A. Additional diagnostics and caveats

Population-level scaling. If $\Phi_{ii}^{(\ell)}$ varies substantially across neurons, the population vector $\Delta\mathbf{A}^{(\ell)}$ is aligned with $-\mathrm{diag}(\Phi_{ii}^{(\ell)})\nabla_{\mathbf{A}^{(\ell)}}\mathcal{L}$, not generally with $-\nabla_{\mathbf{A}^{(\ell)}}\mathcal{L}$; the directions coincide only when the diagonal is approximately uniform.

Appendix B. Annotated layer-2 decomposition

To make the diagonal/off-diagonal split concrete, consider a standard layer-2 block (omitting biases for simplicity):

$$z_i^{(2)} = \sum_m W_{im}^{(2)} A_m^{(1)}, \qquad A_i^{(2)} = \sigma_2\big(z_i^{(2)}\big).$$

The first-order activity update is

$$\Delta A_i^{(2)} \approx -\eta \sum_j \Phi_{ij}^{(2)} \frac{\partial \mathcal{L}}{\partial A_j^{(2)}}.$$

The point of this appendix is to show explicitly where the self term $\Phi_{ii}^{(2)}$ and the cross terms $\Phi_{ij}^{(2)}$ for $j \neq i$ come from.

Own-layer weights contribute only to the diagonal. For a weight $W_{pm}^{(2)}$ feeding directly into layer 2,

$$\frac{\partial A_i^{(2)}}{\partial W_{pm}^{(2)}} = \delta_{ip}\, \sigma_2'\big(z_i^{(2)}\big)\, A_m^{(1)}.$$

Therefore the layer-2 contribution to the kernel is

$$\sum_{p,m} \frac{\partial A_i^{(2)}}{\partial W_{pm}^{(2)}} \frac{\partial A_j^{(2)}}{\partial W_{pm}^{(2)}} = \delta_{ij}\, [\sigma_2'\big(z_i^{(2)}\big)]^2\, \|\mathbf{A}^{(1)}\|^2 = \delta_{ij} D_{2,ii}.$$

This term is diagonal because two different neurons in the same layer do not share their own incoming weights.

Earlier-layer weights are propagated forward by the Jacobian. Now take a weight W_α with $\alpha \in \mathcal{P}_1$. By the chain rule,

$$\frac{\partial A_i^{(2)}}{\partial W_\alpha} = \sum_m \frac{\partial A_i^{(2)}}{\partial A_m^{(1)}} \frac{\partial A_m^{(1)}}{\partial W_\alpha} = \sum_m J_{2,im} \frac{\partial A_m^{(1)}}{\partial W_\alpha}, \qquad J_{2,im} = \frac{\partial A_i^{(2)}}{\partial A_m^{(1)}}.$$

Summing over all layer-1 weights gives

$$\sum_{\alpha \in \mathcal{P}_1} \frac{\partial A_i^{(2)}}{\partial W_\alpha} \frac{\partial A_j^{(2)}}{\partial W_\alpha} = \sum_{m,n} J_{2,im}\, \Phi_{mn}^{(1)}\, J_{2,jn}.$$

Because layer 1 has no earlier shared weights, $\Phi^{(1)} = D_1$ is diagonal, so this reduces to

$$\sum_{m,n} J_{2,im}\, \Phi_{mn}^{(1)}\, J_{2,jn} = \sum_m J_{2,im}\, D_{1,mm}\, J_{2,jm}.$$

Thus the full layer-2 kernel is

$$\Phi_{ij}^{(2)} = \delta_{ij} D_{2,ii} + \sum_m J_{2,im}\, D_{1,mm}\, J_{2,jm}.$$

If one writes the Jacobian explicitly,

$$J_{2,im} = \sigma_2'\big(z_i^{(2)}\big) W_{im}^{(2)},$$

so

$$\Phi_{ij}^{(2)} = \delta_{ij}[\sigma_2'\big(z_i^{(2)}\big)]^2 \|\mathbf{A}^{(1)}\|^2 + \sigma_2'\big(z_i^{(2)}\big)\, \sigma_2'\big(z_j^{(2)}\big) \sum_m W_{im}^{(2)} D_{1,mm} W_{jm}^{(2)}.$$

Self and cross terms in the activity update. Substituting the layer-2 kernel into the first-order update and separating the $j = i$ and $j \neq i$ pieces gives

$$\Delta A_i^{(2)} \approx -\eta\, \Phi_{ii}^{(2)} \frac{\partial \mathcal{L}}{\partial A_i^{(2)}} - \eta \sum_{j \neq i} \Phi_{ij}^{(2)} \frac{\partial \mathcal{L}}{\partial A_j^{(2)}}.$$

Using the expression above,

$$\Phi_{ii}^{(2)} = D_{2,ii} + \sum_m J_{2,im}^2 D_{1,mm},$$

while for $j \neq i$,

$$\Phi_{ij}^{(2)} = \sum_m J_{2,im} D_{1,mm} J_{2,jm}.$$

Hence

$$\Delta A_i^{(2)} \approx -\eta \Big(D_{2,ii} + \sum_m J_{2,im}^2 D_{1,mm} \Big) \frac{\partial \mathcal{L}}{\partial A_i^{(2)}} - \eta \sum_{j \neq i} \Big(\sum_m J_{2,im} D_{1,mm} J_{2,jm} \Big) \frac{\partial \mathcal{L}}{\partial A_j^{(2)}}.$$

The first bracket is the self term: it is always aligned with $-\partial \mathcal{L}/\partial A_i^{(2)}$. The second sum is the interference term: it mixes neuron i with the gradients of other neurons in the same layer through shared earlier-layer parameters.

Relation to the temporary x_0/x_1 notation. If one writes $x_0 = \mathbf{A}^{(0)}$ and $x_1 = \mathbf{A}^{(1)}$, then

$$D_{1,mm} = [\sigma_1'\big(z_m^{(1)}\big)]^2 \|x_0\|^2, \qquad D_{2,ii} = [\sigma_2'\big(z_i^{(2)}\big)]^2 \|x_1\|^2.$$

So the appearance of both x_0 and x_1 in scratch work is meaningful, not a typo: they are simply the activities entering layers 1 and 2. In the main text, $\mathbf{A}^{(0)}$ and $\mathbf{A}^{(1)}$ are cleaner and more consistent with the rest of the notation.

Appendix C. General recursion for deeper networks and nonlinearities

For an arbitrary depth-L feedforward network with pointwise nonlinearities,

$$z_i^{(\ell)} = \sum_m W_{im}^{(\ell)} A_m^{(\ell-1)}, \qquad A_i^{(\ell)} = \sigma_\ell\big(z_i^{(\ell)}\big), \qquad \ell = 1, \ldots, L,$$

again omitting biases for simplicity. The first-order kernel relation from the main text still holds:

$$\Delta \mathbf{A}^{(\ell)} \approx -\eta\, \Phi^{(\ell)}\, \nabla_{\mathbf{A}^{(\ell)}} \mathcal{L}, \qquad \Phi_{ij}^{(\ell)} = \sum_{\alpha \in \mathcal{P}_{\leq \ell}} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial A_j^{(\ell)}}{\partial W_\alpha}.$$

To obtain a recursion, split the weight sum into layer ℓ itself and layers $1, \ldots, \ell - 1$.

Own-layer contribution. For a weight $W_{pm}^{(\ell)}$ in layer ℓ,

$$\frac{\partial A_i^{(\ell)}}{\partial W_{pm}^{(\ell)}} = \delta_{ip}\, \sigma_\ell'\big(z_i^{(\ell)}\big)\, A_m^{(\ell-1)}.$$

Therefore the own-layer contribution is diagonal:

$$\sum_{p,m} \frac{\partial A_i^{(\ell)}}{\partial W_{pm}^{(\ell)}} \frac{\partial A_j^{(\ell)}}{\partial W_{pm}^{(\ell)}} = \delta_{ij} [\sigma_\ell'\big(z_i^{(\ell)}\big)]^2 \|\mathbf{A}^{(\ell-1)}\|^2 = \delta_{ij} D_{\ell,ii}.$$

Earlier-layer contribution. Define the inter-layer Jacobian

$$J_{\ell,im} = \frac{\partial A_i^{(\ell)}}{\partial A_m^{(\ell-1)}} = \sigma_\ell'\big(z_i^{(\ell)}\big) W_{im}^{(\ell)}.$$

Then for any weight W_α with $\alpha \in \mathcal{P}_{\le \ell-1}$,

$$\frac{\partial A_i^{(\ell)}}{\partial W_\alpha} = \sum_m J_{\ell,im} \frac{\partial A_m^{(\ell-1)}}{\partial W_\alpha}.$$

Summing over earlier-layer weights gives

$$\sum_{\alpha \in \mathcal{P}_{\le \ell-1}} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial A_j^{(\ell)}}{\partial W_\alpha} = \sum_{m,n} J_{\ell,im}\, \Phi_{mn}^{(\ell-1)}\, J_{\ell,jn}.$$

Combining both pieces yields the recursion

$$\Phi^{(\ell)} = D_\ell + J_\ell\, \Phi^{(\ell-1)}\, J_\ell^T, \qquad \Phi^{(1)} = D_1.$$

This is the desired recursion in expanded form.

Unrolling the recursion. Repeated substitution shows that every earlier layer contributes a positive-semidefinite term transported forward by the intermediate Jacobians:

$$\Phi^{(\ell)} = D_\ell + J_\ell D_{\ell-1} J_\ell^T + J_\ell J_{\ell-1} D_{\ell-2} J_{\ell-1}^T J_\ell^T + \cdots + J_\ell J_{\ell-1} \cdots J_2\, D_1\, J_2^T \cdots J_{\ell-1}^T J_\ell^T.$$

This makes the structure transparent: D_r captures the direct effect of layer-r weights on layer-r activities, and the surrounding Jacobians propagate that contribution forward to layer ℓ.

Role of nonlinearities. Nothing in the kernel definition or recursion requires linear hidden units. For smooth nonlinearities such as sigmoid, tanh, or GELU, the algebraic derivation above holds with the corresponding σ_ℓ'; the overall $\Delta\mathbf{A} \approx -\eta\Phi\nabla_{\mathbf{A}}\mathcal{L}$ remains a first-order approximation in η. For piecewise-linear nonlinearities such as ReLU or leaky ReLU, the same formulas hold almost everywhere; at kink points one may use a subgradient. In particular, if a ReLU unit is inactive on the sampled input, then $\sigma_\ell'\big(z_i^{(\ell)}\big) = 0$, so its row of J_ℓ and its diagonal entry in D_ℓ vanish. This is the appendix-level version of why dead ReLU units contribute nothing to the kernel on that input.

More general feedforward modules. The recursion above assumes an affine layer followed by a pointwise nonlinearity. The kernel definition

$$\Phi_{ij}^{(\ell)} = \sum_{\alpha \in \mathcal{P}_{\leq \ell}} \frac{\partial A_i^{(\ell)}}{\partial W_\alpha} \frac{\partial A_j^{(\ell)}}{\partial W_\alpha}$$

and the first-order relation $\Delta \mathbf{A}^{(\ell)} \approx -\eta\, \Phi^{(\ell)} \nabla_{\mathbf{A}^{(\ell)}} \mathcal{L}$ still hold for any differentiable sequential feedforward architecture (i.e. one without skip connections). For other modules the same logic applies: there is an own-module contribution from parameters in layer ℓ and a propagated earlier-layer contribution obtained by sandwiching $\Phi^{(\ell-1)}$ between the appropriate Jacobians. Only the explicit forms of D_ℓ and J_ℓ change.